

Conversation Export

- Project: Medical_app_V3
- Topic: blood pressure model debugging, AWS deployment planning, Docker backend/AI packaging, and EC2 deployment steps.
- Date: 2026-07-01

1. Blood Pressure Model Check

- User asked: the blood pressure model doesnt work, check it.
- Finding: the BP Keras model artifacts were present and direct inference worked with synthetic PPG plus patient height/weight metadata.
- Direct result observed: about 124.4 / 66.9 mmHg.
- Actual issue found: backend analysis could call the BP model with patient=None when a monitoring session was created before the device was assigned to a patient.
- Fix made: session analysis now resolves the patient from session.patient first, then falls back to session.device.patient, and backfills session.patient_id when possible.
- Files touched: Backend-bisho/routers/ecg_pipeline.py, Backend-bisho/app/services/analysis_worker.py, Backend-bisho/tests/test_ecg_pipeline_api.py.
- Validation: tests/test_ecg_pipeline_api.py and tests/test_ai_integration.py passed, 24 tests total.
- Remaining requirement: BP still needs valid patient height/weight and usable PPG pulse peaks.

2. AWS Cluster Deployment Review

- User asked what must change to deploy the project as a cluster on AWS, including IP addresses.
- Recommendation: use ECS Fargate plus Application Load Balancer plus RDS PostgreSQL unless Kubernetes/EKS is specifically required.
- Backend public URL should move from local/LAN addresses to a stable DNS name such as https://api.your-domain.com.
- Flutter should be built with --dart-define=BACKEND_BASE_URL=https://api.your-domain.com.
- ESP32 firmware should use BACKEND_READINGS_URL=https://api.your-domain.com/api/readings.
- Backend env vars should be set in AWS Secrets Manager, SSM, ECS task env, or Kubernetes Secret, not committed .env files.
- Important env vars: DATABASE_URL, SECRET_KEY, DEVICE_INGEST_TOKEN, CORS_ORIGINS, GEMINI_API_KEY, ECG_MODEL_DIR, MORPHOLOGY_MODEL_DIR, BP_MODEL_DIR.
- Database should be RDS PostgreSQL in private subnets/security groups, not a public database.
- Security groups: public ALB on 443, backend tasks on 8000 only from ALB, RDS on 5432 only from backend.
- AI model artifacts must be baked into the Docker image, mounted from EFS, or downloaded from S3 at startup.
- Cluster warning: the in-memory analysis queue is not durable and each replica has its own worker. For true multi-replica production, split API and worker and use SQS, Redis/Celery, or database-backed jobs.

3. Dockerfile Request

- User first asked for a Dockerfile for the entire project.
- Initial direction considered: one image serving Flutter web through nginx and proxying API to FastAPI.
- User clarified: they wanted a Dockerfile for AI models and backend only, to deploy on an AWS instance.
- Final result: root Dockerfile packages Backend-bisho plus AI models into a Python 3.11 slim image.
- The Dockerfile installs backend requirements, copies Backend-bisho into /app, copies AI models into /app/ai_models, exposes port 8000, and runs uvicorn.
- The image sets model env defaults:
ECG_MODEL_DIR=/app/ai_models/ecg_classifier_v31_deployment,
MORPHOLOGY_MODEL_DIR=/app/ai_models/Morphological Heartbeat Arrhythmia Classifier,
BP_MODEL_DIR=/app/ai_models/BP_Model_Vital_15_Meta.
- Added .dockerignore to keep local env, build outputs, Python caches, Flutter artifacts, and hardware secrets out of Docker context.
- Alembic env.py was adjusted to honor DATABASE_URL and DATABASE_PORT, important for AWS RDS deployments.
- Docker build was not run because it would need network pulls for base images and packages.

4. Current Backend and AI Dockerfile

- Build from project root:
- docker build -t medical-backend-ai .
- Run locally or on EC2:
- docker run --rm -p 8000:8000 -e DATABASE_URL=postgresql://user:password@your-rds-endpoint.amazonaws.com:5432/medical_app -e SECRET_KEY=replace-with-long-secret -e DEVICE_INGEST_TOKEN=replace-with-device-token -e CORS_ORIGINS=https://your-app-domain.com -e RUN_MIGRATIONS=true medical-backend-ai
- For AWS, expose container port 8000. If using a domain, place nginx or an AWS load balancer in front for HTTPS.

5. EC2 Deployment Steps

- User asked what they need to do to deploy backend and AI model on an EC2 instance.
- Recommended EC2: Ubuntu, t3.medium or t3.large, 30-50 GB disk.
- Security group: open SSH 22 only from your IP; open 8000 temporarily for API testing or use 80/443 with nginx; do not expose PostgreSQL publicly.
- Install Docker: sudo apt update; sudo apt install -y docker.io git; sudo systemctl enable docker; sudo systemctl start docker; sudo usermod -aG docker ubuntu; then log out and SSH again.
- Copy project to EC2 by git clone or SCP. Ensure Backend-bisho, AI models, Dockerfile, and .dockerignore are present.
- Create medical.env on EC2 with DATABASE_URL, SECRET_KEY, DEVICE_INGEST_TOKEN, CORS_ORIGINS, GEMINI vars, and RUN_MIGRATIONS=true for first startup.
- Build image: docker build -t medical-backend-ai .
- Run container: docker run -d --name medical-backend --restart unless-stopped --env-file medical.env -p 8000:8000 medical-backend-ai.
- Check logs: docker logs -f medical-backend.
- Test API: curl http://YOUR_EC2_PUBLIC_IP:8000/.

- Hardware URL should become `http://YOUR_EC2_PUBLIC_IP:8000/api/readings` for temporary testing, or `https://api.your-domain.com/api/readings` for production.
- Flutter should use `--dart-define=BACKEND_BASE_URL=http://YOUR_EC2_PUBLIC_IP:8000` or the HTTPS domain.
- For real production, use HTTPS, a domain, and preferably RDS PostgreSQL rather than a database exposed on the EC2 public network.